第7章 第2节 MCP 核心概念

- 来源: https://class.imooc.com/lesson/5278#mid=104931
- 阶段: 阶段三: 【工具与协议】让AI连接世界
- 周: 第7周 MCP：模型上下文协议
- 模块: MCP核心概念
- 课节: 图文 1-2 MCP 核心概念

1-2 MCP 核心概念

第7章 第2节 MCP 核心概念

> [!TIP] 阅读指南 本节先对比传统 Function Calling 与 MCP 的架构差异，再拆解 MCP Server 的三大核心能力（Resources、Tools、Prompts），最后梳理 Client-Server-LLM 三方的分工关系。 上一节介绍了 MCP 的诞生背景和要解决的问题。接下来深入核心，看 MCP 到底是如何设计的，以及它的完整工作流程。 MCP 的设计与 Function Calling 有一个根本不同：Function Calling 的目的是「让 AI 干活」，目标直观；MCP 在很大程度上是围绕安全性设计的——权限边界、进程隔离、数据访问权归属。这些概念不像调用函数那样一看就懂，需要你有一点安全意识，才能体会其中的设计巧思。所以这一章不用急，静下心，一点点看进去。

2.1 从 Function Calling 到 MCP

在第4章里，实现过一个典型的 Function Calling 流程，本质上可以拆成下面几步：

```
你在应用里自己写工具函数
↓
通过 tools 暴露给 LLM
↓
LLM 决定调用哪个函数并给出参数
↓
你的代码实际执行函数并把结果回传给 LLM
↓
LLM 基于结果生成最终回答
```

换句话说，工具的定义、执行和结果处理全部耦合在应用进程里，每个应用都要重复实现一遍。而 MCP 的思路，是把这部分能力"抽出去"，做成可以被多个应用共享的标准化工具服务。

具体来说，MCP 把"工具"这部分独立出来，变成一个单独的进程：

单独的进程

核心变化是：工具不再写在代码里，而是由独立的 MCP Server 提供；AI 应用变成 MCP Client，通过协议与 Server 通信。

用一个简单的例子来看 MCP 的工作方式——用户让 AI 查询天气，这显然需要调用天气查询工具：

简化流程

先用一个简化的时序图理解核心流程：

核心流程

- 获取工具：Client 从 Server 获取可用工具列表（get_weather、get_time 等）
- 决策：LLM 理解用户意图，决定调用 get_weather 工具

决策

- 执行：Server 执行工具（Client 转发 LLM 的调用请求），获取实际天气数据并返回
- 回复：LLM 基于天气数据生成自然语言回复

回复

> 重点： MCP Server并不决定使用哪个工具，只是提供工具列表给LLM进行选择，最终是LLM决定使用哪个工具。

MCP vs Function Calling

MCP与Function Calling在机制上有很大的不同：

- MCP的思想类似从手机的应用中心里下载应用程序（工具），而Function Calling的工具是直接写在AI应用里的。
- Function Calling的工具大多数时候和AI应用紧密耦合，而MCP的工具可以独立出来，让多个AI应用共享。
- Function Calling的工具往往和AI应用运行在同一个进程里，安全问题突出——恶意用户写一个工具就能删除文件。MCP通过将工具运行在独立进程中，天然规避了这个问题。

如果还是不太好理解，可以把MCP类比成Python通过pip安装各种包：需要的时候访问Server获取工具，而不需要自己实现。

为什么这样设计

独立进程带来的好处：工具运行在独立进程，崩溃不会影响 AI 应用；可以通过进程级别的权限隔离来限制工具能力；Server 可以用任何语言实现，Python、Node.js、Go 都行。

2.2 MCP Server 能提供什么

MCP并没有前面讲的那么简单。前面为了简化理解，将MCP Server类比成了应用中心（AppStore）。但它远比一个简单的工具商店复杂——MCP Server 其实提供了三种核心能力，逐一来拆解。

在前面的流程图中，看到 AI 应用通过 MCP Server 获取工具、请求执行操作，由 Server 完成实际执行。那 MCP Server 到底能提供什么？答案是三种核心能力：

可能看的有点懵。没关系，通过一个场景比喻来理解这三者的区别。

通过场景理解三大能力

假设在用一个 AI 阅读助手，帮自己阅读《红楼梦》：

查询小说内容

```
用户："林黛玉第一次见到贾宝玉是在哪一回？"

1
```

→ AI 需要读取小说文本（Resources）

读取小说文本

添加书签

```
用户："帮我在第27回'宝钗扑蝶'这里添加一个书签"

1
```

→ AI 需要执行创建书签操作（Tools）

执行创建书签操作

询问使用方法

```
用户："你能帮我做什么？"

1
```

→ AI 可以使用预设的使用说明模板（Prompts）

预设的使用说明模板

对应MCP的能力

场景	需要的能力	特点
查询小说内容	Resources	只读数据，无副作用
添加书签	Tools	执行操作，会改变状态
使用说明	Prompts	标准化的交互模板

MCP 的工作流程

在这个流程里，有一个很关键但容易被忽略的问题：到底是谁决定 AI 应该使用哪个能力？ 换句话说，在这个例子中，是谁在什么时机决定“读取哪个 Resource、调用哪个 Tool”？ 理解这一点，才能真正看清 MCP 中 Client、Server 和 LLM 之间的分工。

关键角色分工

角色	职责	不负责
LLM	决策使用哪个 Resource/Tool	不直接调用 MCP Server
AI 应用	编排调度：发协议请求、收结果、喂给 LLM	不决策用哪个
MCP Server	提供 Resources/Tools/Prompts	不参与决策

一点也不意外，还是LLM这个最强大脑来做决策，决定到底调用哪个工具。

AI 不直接和 MCP Server 打交道

- LLM 只是决策者：它只负责理解用户意图，决定该用哪个工具

LLM 只是决策者

- AI 应用是调度者：它负责协议通信和流程编排——对 Server 发请求、拿结果、传给 LLM
- 分工明确：LLM 专注于智能决策，AI 应用处理技术细节

分工明确

- MCP Server：像一个服务中心，提供 Resources/Tools/Prompts，不参与决策

到这里可能还是有疑惑，Resources 和 Tools 到底是什么？我能理解工具，但我不能理解 Resources。可以先有一个大致的概念：Tools 更接近我们在第4章里说的“函数工具”，负责“做事”；而 Resources 更像一份“只读的数据源”，负责“提供信息”——比如一整本小说、一个文档库、一个数据库表，下一节我们会专门用一整节来把 Resources 讲清楚。

至于为什么要把 Resources 和 Tools 分开，本节不展开细节，下一节在专门讲 Resources 的时候会结合更多实战场景来系统解释这个设计动机。

2.3 下一节预告

本节从整体架构上认识了 MCP：它如何在 Client-Server 模型下组织 Resources、Tools 和 Prompts，以及 LLM、AI 应用和 MCP Server 之间的分工关系。现在可以把 MCP 看成是"把工具和数据源统一包装成服务"的一层，但这些抽象还比较粗。

接下来，在第5章第3节中，会把视角收缩到其中的一条能力——Resources：深入讲清它到底长什么样、如何帮助 AI 精准找到相关文档，以及它和传统 RAG 检索之间的异同与配合方式，帮读者理解如何用 MCP 来管理知识与数据。

2.4 ■ 学点英语

中文	English	音标	说明
三方架构	Three-Party Architecture	/θriː ˈpɑːrti ˈɑːrkɪtektʃər/	LLM + AI应用 + MCP Server 各司其职的分工架构
能力声明	Capabilities Declaration	/ˌkeɪpəˈbɪlətiz ˌdekləˈreɪʃn/	初始化阶段Server向Client声明支持的MCP功能列表
进程隔离	Process Isolation	/ˈprɑːses ˌaɪsəˈleɪʃn/	MCP Server运行在独立进程中，与AI应用互不影响的隔离机制
服务编排	Service Orchestration	/ˈsɜːrvɪs ˌɔːrkɪˈstreɪʃn/	AI应用在LLM和MCP Server之间协调消息传递的过程

图片



